Automated Plant disease detection: CNN for corn maize, tomato and potato

Angeline R[1] and Aruneshwaran S[2]

[1,2]Department of Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning
SRM Institute of Science and Technology, Ramapuram

Abstract. Plant ailments pose present a significant challenge to the worldwide food security and the agricultural sector. Swift and precise detection of these diseases is pivotal for effectively managing them and preventing crop yield reductions. Lately, advanced deep learning techniques, specifically Convolutional Neural Networks (CNNs), have exhibited encouraging outcomes across various tasks involving image recognition. This undertaking strives to create and execute a model founded on CNNs to prognosticate plant diseases through leaf images. The proposed strategy encompasses three main phases: compiling and preparing the data, developing the model architecture, and assessing performance. Initially, an extensive dataset of plant leaf images, encompassing leaves afflicted by diverse diseases, is assembled. The images undergo preprocessing to heighten quality and eliminate disturbances, ensuring a dependable model training process. Subsequently, a CNN structure is devised and trained employing the dataset. The chosen CNN model adheres to a sequential design, where each layer possesses precisely one input and output. These layers are arranged sequentially to construct the entire network and incorporate multiple convolutional layers such as Conv2D, MaxPooling2D, Flatten, and Dense, enabling the learning of features from the input images. The findings underscore that the CNN-centered model for forecasting plant diseases attains remarkable training precision of 99.65%, accompanied by a testing precision of 99.44% and a validation precision of 98.61%, proficiently identifying prevalent ailments like common rust disease in corn plants, bacterial spot infection in tomato crops, and the early blight ailment in potato plants. In conclusion, the proposed CNN-driven prognostic model for plant diseases manifests encouraging outcomes in precisely recognizing these diseases from leaf images. The efficacious application of this model can assist farmers and agricultural specialists in informed decision-making, ultimately leading to enhanced crop administration, decreased losses, and a positive impact on global food security. Furthermore, this endeavor establishes a groundwork for forthcoming research and breakthroughs in the realm of deep learning-assisted plant disease detection.

Keywords: Image categorization, (ML) Machine Learning, (CNN) Convolutional Neural Network, (DL) Deep Learning, Plant disease identification

1 Introduction

There is a proverb saying that “Prevention is better than cure” and this is the project motto as well. Plant diseases are one of the foremost obstacles confronting modern agriculture, affecting crop yield, quality, and worldwide food safety. Timely and accurate recognition of these ailments holds a central role in averting devastating consequences and minimizing economic losses for farmers and agricultural industries. Traditionally, the recognition of plant diseases used to depend on manual examination. by trained experts, which is labor – intensive, utilizes substantial portion

of time and frequently susceptible to human inaccuracies. However, recent advancements in computer vision and DL techniques have cleared the way for automating and improving disease detection processes. CNNs have risen as a potent instrument in image identification tasks and have showcased remarkable achievements in diverse, such as object detection, facial recognition, and natural language processing. Their capacity to grasp intricate patterns and hierarchies of attributes from raw information makes them particularly suitable for image-based tasks, including plant disease prediction.

This project aims to harness the potential of CNNs to create a streamlined and precise plant disease detection system. The project uses 300 images of potato plant leaf affected with early blight, 300 images of tomato leaf affected bacterial spot and 300 images of corn maize leaf affected with common rust so, 900 images in total and is taken from the new plant diseases dataset from Kaggle. The proposed system seeks to distinguish between various plant disease states accurately.

The importance of this research is underscored by its capacity to revolutionize plant disease management practices. By providing an automated solution, the proposed system could enable early detection of diseases, enabling farmers and agricultural experts to respond promptly with targeted interventions such as disease control measures, optimized irrigation, and precision pesticide application.

In this paper, the detailed methodology employed to build the plant disease prediction system using CNNs. The steps involved in preparing the dataset, including data augmentation techniques to augment the training data and improve the model's generalization are described. Additionally, the architecture of the CNN model designed for classification of plant diseases is outlined.

The performance of the proposed system is rigorously evaluated using a set of plant species and disease types and the project is deployed using Streamlit where the image can be submitted and find the disease it has been affected with. Furthermore, the implications and potential applications of this system in real-world agricultural scenarios, emphasizing its role in enhancing overall crop productivity and contributing to sustainable farming practices are discussed.

In conclusion, this project represents a notable stride in harnessing the capabilities of deep learning to tackle the obstacles in plant disease detection. By automating the process and enabling early disease identification, this system has the potential to reshape agricultural practices and foster a more sustainable and resilient food production system in the face of increasing global demands and environmental changes.

2 Related Works

[1] As indicated by this study, the researchers employed the Plant Village Dataset and assessed three primary neural network architectures and they are, Faster Region-based Convolutional Neural Network (Faster R-CNN), Shot Multi Box Detector (SSD) and Region based Fully Convolutional Neural Network (R-CNN). These evaluations resulted in an accuracy of 94.6%. The problem in this method is on using the Faster R-CNN requires a high training time, R-CNN is inefficient at handling aspect ratios and

SSD gives lower accuracy for large objects.

[2] In this analysis, a multitude of sample images of soybean plants were employed, encompassing leaf images from four distinct classes: septorial leaf blight, downy mildew, frogeye leaf spot and healthy leaf images. These images were sourced from the PlantVillage database. The researchers adopted the LeNet architecture for conducting disease classification. As a result of this approach, an impressive accuracy rate of 99.32% was attained. The issue here is on using the LeNet which is relatively shallow network compared to modern CNN architectures so it is prone to overfitting and also lacks batch normalization and regularization and was designed in early 1990s so, it is not suitable for modern data.

[3] In the present research, a modified version of the LeNet CNN model was employed to identify and detect diseases in tomato leaves. The researchers utilized the PlantVillage dataset to train the model. This innovative system accomplished an average accuracy ranging from 94% to 95%. These results highlight the viability of the neural network methodology, even when dealing with challenging conditions. The problem in this method is, On using the LeNet which is relatively shallow network compared to modern CNN architectures so it is prone to overfitting and also lacks batch normalization and regularization and was designed in early 1990s so, it is not suitable for modern data.

[4] In this particular study, the researchers employed the publicly accessible PlantVillage dataset to acquire leaf images from peach plants. They developed a hybrid model that is a combination of a Convolutional Autoencoder (CAE) network with a Convolutional Neural Network (CNN) to facilitate automatic identification of plant diseases. Remarkably, this methodology yielded a test precision of 98.38% and a training precision of 99.35%. The issue in this method is, CAEs are often trained in unsupervised manner, which means they may not fully capture higher-level semantic information and lacks hierarchial learning. Selecting an appropriate CNN architecture for a give task can be challenging.

[5] This study introduces a method for identifying the plant diseases using CNN. The research involves conducting a simulation study and analyzing sample images to assess the time complexity and the extent of the infected region. The analysis employs image processing techniques. The dataset used for evaluation comprises a total of 15 cases. Among these, 12 cases pertain to diseased plant leaves. Additionally, the dataset includes 3 cases of healthy leaves and the CNN model achieved a test precision of 88.80% in disease classification. The problem here is, CNNs are data-hungry and computationally intensive. Training deep CNNs often requires large datasets and powerful hardware, making them inaccessible to smaller organizations or individuals.

[6] As Explored within this research are two distinct methods, namely the utilization of a shallow VGG model coupled with Random Forest (RF) and a shallow VGG model combined with Xgboost. These methods are employed for the purpose of disease identification. The model is compared against alternative techniques, both custom-made and deep learning-based, in experiments conducted on the three various different plant types which are: corn, tomato and potato. Within the realm of corn, the diseases under consideration are Gray leaf spot, Common rust, and Blight. For potatoes, the diseases include late and early blight, while the tomato diseases consist of bacterial spot, late and early blight. The results indicate that the implemented shallow VGG model with Xgboost surpasses various deep learning models across key metrics such as f1-score, precision recall, accuracy and specificity. Notably, the Shallow Visual Geometric Group (VGG) model combined with Xgboost achieves the highest accuracy rates which are as follows, 94.47% for corn and 98.74% for potatoes and 93.91% for the tomato. The issue with this method is, training a shallow VGG model can be time consuming and then combining it with RF or XGBoost might increase the overall training time.

[7] As a part of this study, the PlantVillage dataset was employed. Notably, the utilization of traditional Machine Learning (ML) techniques yielded a commendable accuracy rate of 94.9% using k-Nearest Neighbors (k-NN). Meanwhile, the application of a pretrained VGG16 model resulted in an accuracy of 93.5%. In pursuit of enhancing the performance of the suggested CNN, a strategy of image pre-processing was implemented. This involved adjusting image brightness by a randomly determined value within a variable width range, applied subsequent to image augmentation. Impressively, this modified CNN model with 8 layers also demonstrated exceptional performance beyond the PlantVillage dataset, boasting an accuracy of 98.7%. The problem with this method is, training an 8-layered CNN might take longer time compared to shallower architectures, as the network becomes deeper the vanishing gradient problem can arise.

[8] In the course of this investigation, the researchers made use of the PlantVillage dataset. They devised a strategy involving the fusion of multiple Convolutional Neural Networks (CNNs), which facilitated the development of the proposed UnitedModel. This innovative approach enabled the extraction of complementary and distinctive features. Specifically, the UnitedModel designed to discern grape leaf ailments was built upon two well-known deep learning architectures which are ResNet and GoogLeNet. Impressively, this model attained a noteworthy average validation precision of 99.17%, coupled with a robust test precision of 98.57%. The issue with this method is, GoogLeNet require more memory than other architectures it may entail significant time investment and resource demanding if not implemented efficiently. ResNet architectures require longer training time due to the increased number of layers.

[9] Within the framework of this study, the researchers employed the PlantVillage dataset, specifically focusing on five distinct leaf diseases which are Leaf webber, Anthracnose, Leaf Gall, Alternaria leaf spots, and Leaf burn of Mango. The investigation involved the use of diverse pre-trained CNN architectures, including VGG-19, AlexNet, VGG-16, and a custom deep CNN architecture comprising three hidden layers. The outcome of this endeavor was an remarkable precision rate of 96.67% in effectively identifying the aforementioned leaf diseases in mango plants. The problem in this method is, training VGG models is time consuming due to their depth and convergence can be slow and AlexNet's architecture might not fully exploit the parallel processing capabilities of modern GPUs (Graphics Processing Unit), potentially impacting its training speed. A deep CNN with only 3 hidden layers might have limited capacity to capture complex pattern in data.

[10] In this research, the researchers gathered a diverse range of images from multiple online sources, including the UCI Machine Learning repository and Kaggle. A total of 984 images were collected for analysis. The focus of the research was on various diseases affecting paddy leaves, including brown spot, leaf blight, leaf burst, and leaf smut. Additionally, the dataset encompassed healthy paddy leaf images. The investigation encompassed the evaluation of four distinct models: Xception, VGG-19, Inception-ResNet-V2, and ResNet-101. The standout performance was observed with the Inception-ResNet-V2 model, achieving a notable precision rate of 92.68%. The issue with this method is, VGG-19 demands a significant amount of memory due to its extensive number of layers and parameters. The architecture of Inception-ResNet-V2 is complex, which might make it challenging to implement, fine-tune, and troubleshoot. ResNet-101's depth makes it time-consuming to train, especially on larger datasets, and might require specialized hardware to train efficiently. The intricate architecture of Xception can lead to longer training times compared to shallower models.

3 Dataset

On using the new plant diseases dataset from Kaggle and in that dataset, 300 images of potato affected with early blight, 300 images of tomato affected with bacterial spot and 300 images of corn maize affected with common rust are taken and the reason for taking 300 images in all the three classes is to avoid data imbalance which can cause overfitting so, the dataset is taken as a balanced one. So, a total of 900 images as dataset is taken for the research and those images are visualized and resized if needed.

Fig. 1. Visualizing the images in the dataset.

All the images in the dataset (Fig. 1) have 256 horizontal pixels and 256 vertical pixels that is, they are already in 256 x 256 size so, resizing is not required here.

4 Methodology

A neural network model is in the process of being constructed and will subsequently be deployed on a Streamlit web application. The methodology's specifics encompass several stages, which are visually depicted in the form of a schematic diagram illustrated in Fig. 2.

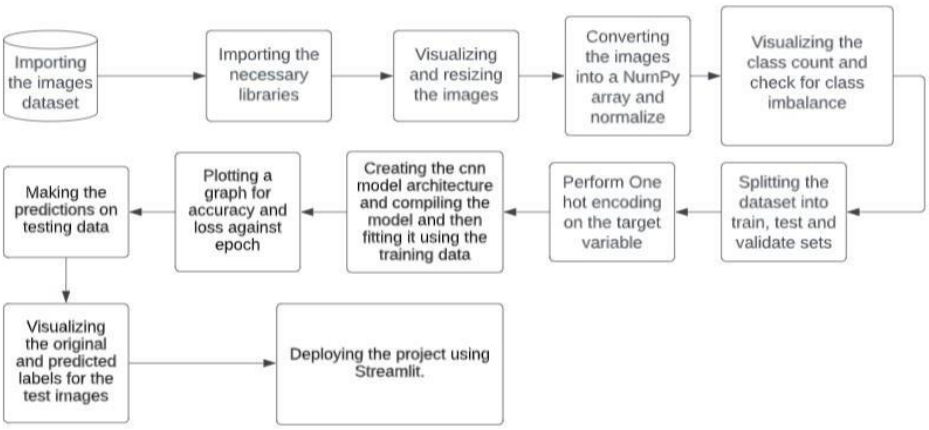

Fig. 2. Schematic diagram for the steps present in the proposed methodology.

- Initiating the process, the initial step involves the collection of data. Subsequently, the collected dataset undergoes pre-processing facilitated by Keras.
- The necessary libraries should be imported in order to serve specific purposes in various aspects of data processing, analysis, visualization, and deep learning model building.
- The images in the dataset are visualized and are resized if the images are not in the size of 256 x 256.

- Converting images to NumPy arrays and normalizing them are essential preprocessing steps that prepare image data for use in a plant disease prediction project that is based on CNN. Normalization is done where the pixel values are in the range [0 , 1] is divided for each pixel value by 255. This scales the values to a floating-point range where 0.0 depicts the absence of the color, and 1.0 depicts the maximum intensity of that color. These steps ensure that the model can effectively learn from the data and make accurate predictions.

- The class count for the three different classes is visualized and the balance of the class is been checked in order maintain the same class count across all the three different classes and this step ensures that no overfitting occurs.

- At the outset, the dataset is divided into 20% testing data and 80% training data and then the training data is once again divided for data validation.

- Performing one-hot encoding on the target variable is a common preprocessing step in ML and DL tasks, especially for classification problems. One-hot encoding is a method employed for expressing categorical data, such as class labels, in a format that machine learning algorithms can understand. It transforms categorical values into a binary matrix, where each category is portrayed as a binary vector, wherein a 1 occupies the position corresponding to the category, while the rest of the positions contain 0s elsewhere. This encoding ensures that each class is represented uniquely, and the model can learn from these binary vectors during training.

- The CNN model architecture is created and is compiled and then it is fitted using the training data and then the created model is saved.

- Assess the trained model and is tested on the evaluation dataset to gauge its performance. The model will produce a probability distribution over the classes. The predicted class will be determined by selecting the class with the highest probability and the output will be produced accordingly.

- The deployment of the model will take place on the Streamlit web application.

4.1 Reason for choosing Sequential Convolution Neural Network Architecture:

The choice of a Sequential CNN architecture over other complex architectures like Faster R-CNN, R-CNN, SSD, LeNet, CAE, Shallow VGG, GoogLeNet, VGG-19, VGG-16, AlexNet, Xception, ResNet-101, and Inception-ResNet-V2 is due to the following reasons,

- Sequential CNNs are relatively simple and straight forward to design, implement, and train. They have fewer layers and components compared to complex architectures like Inception-ResNet-V2 or Faster R-CNN

- More complex architectures like SSD and Faster R-CNN typically require large amounts of annotated object bounding boxes, which can be challenging to obtain in plant disease detection datasets. If the labeled data is limited and cannot perform object localization, a Sequential CNN can still provide valuable classification results.
- Complex architectures often require significantly more computational resources (both in terms of training time and hardware requirements) than Sequential CNNs. If the computational resources is limited or need to deploy the model on edge devices with resource constraints, a simpler model will be a better choice.
- If the primary task is image classification (i.e., determining the disease category of a plant based on an input image), then a Sequential CNN will be a suitable choice. Complex architectures like Faster R-CNN and SSD are designed for object detection and localization, which is a more involved task.
- If the model is to be deployed in real-world scenarios, simpler architectures are often easier to deploy and maintain and it has lower inference time requirements, making them suitable for real-time or near-real-time applications.

4.2 Sequential Convolution Neural Network Architecture:

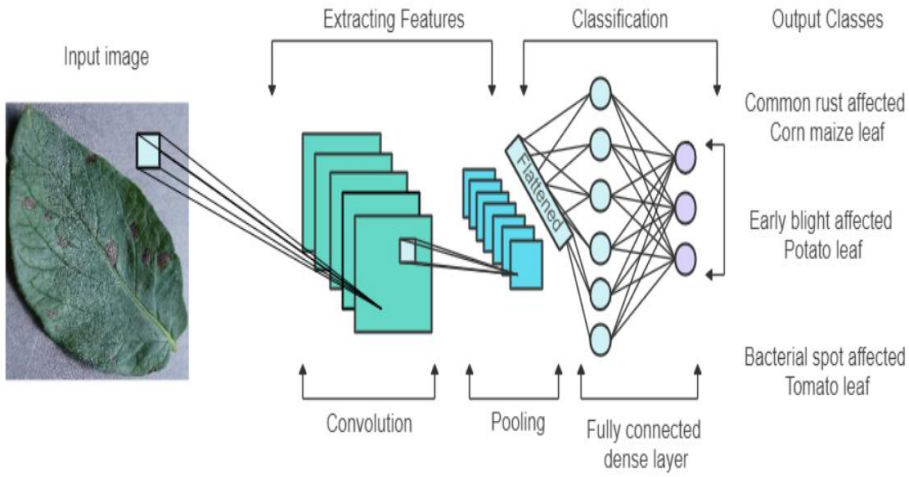

Fig. 3. Illustration depicting the architecture of a Sequential Convolution Network.

The architecture of the neural network model (Fig. 3) was established, incorporating distinct layers to correspond with their respective features. These layers encompass Conv2D which is the convolution layer, MaxPooling2D which is the pooling layer, Flatten, and Dense, each serving a distinct purpose. Convolution layer is the initial stage in the process of extracting dimensions from input images is the convolutional layer. This layer employs filters to capture specific attributes, leading to the creation

of a feature map based on the input images. This operation is fundamentally mathematical and involves the interaction of two inputs. Conv2D is employed to generate a convolutional kernel that convolves with the input layer, yielding the output tensor. The pooling layer operates by using a 2D filter that scans across each channel of the feature map, capturing the characteristics present within the filter's covered region. The max-pooling layer selects and retains the highest values within a specific region of the feature map. Consequently, the max-pooling layer produces a output which is a new feature map that highlights the most significant features from the previous one and the MaxPooling2D operates as a down sampling technique by extracting maximum values within a defined window size. The fully connected layer within a CNN serves as the feature vector representation of the input, encapsulating crucial information. It is instrumental in determining the loss, aiding the network's training process. The convolutional layers preceding the fully connected layers contain essential details about local features within the input image, such as edges, patterns, and shapes. Each convolutional layer comprises multiple filters, each representing a distinct local feature. In contrast, the fully connected layer consolidates and integrates the data from all preceding convolutional layers, emphasizing the most significant composite details. Flatten serves to transform the input into a one-dimensional output, and Dense generates output through the dot product of the input and the kernel. To ensure optimal model performance, the choice of loss function should be Categorical Crossentropy since there are more than two disease categories for prediction, while the optimizer should be set to Adam, and the metrics to accuracy. This successful configuration culminates in the creation of the network architecture for the model. The variations in model summaries with distinct parameter values in Table I and Table II.

Type of layer	Shape of output	No. of Parameters
Conv_2D	(None, 256, 256, 32)	896
MaxPooling_2D	(None, 85, 85, 32)	0
Conv_2D-1	(None, 85, 85, 16)	4624
MaxPooling_2D-1	(None, 42, 42, 16)	0
Flatten	(None, 28224)	0
Dense	(None, 8)	225800
Dense-1	(None, 3)	27

TABLE I. MODEL SUMMARY 1

No. of total parameters	2,31,347
No. of trainable parameters	2,31,347
No. of non – trainable parameters	0

TABLE II. MODEL SUMMARY 2

5 Result

During the model training phase, remarkable accuracy metrics were attained. The training accuracy reached an impressive 99.65%, while the validation accuracy was registered at 98.61% across 50 epochs (see Fig. 4).

```
5/5 [==============================] - 38s 7s/step - loss: 0.0258 - accuracy: 0.9965 - val_loss: 0.0433 - val_accuracy: 0.9861
Epoch 49/50
5/5 [==============================] - 38s 8s/step - loss: 0.0255 - accuracy: 0.9965 - val_loss: 0.0384 - val_accuracy: 0.9861
Epoch 50/50
5/5 [==============================] - 36s 7s/step - loss: 0.0243 - accuracy: 0.9965 - val_loss: 0.0385 - val_accuracy: 0.9861
```

Fig. 4. Accuracy achieved at the 50th epoch.

Subsequently, when the model was subjected to testing, a high test accuracy of 99.44% was achieved (as shown in Fig. 5).

```
Calculation of the model accuracy
6/6 [==============================] - 5s 865ms/step - loss: 0.0416 - accuracy: 0.9944
Test Accuracy: 99.44444298744202
```

Fig. 5. Calculation of the Test Accuracy.

To provide further insight, visualizations were generated showcasing both the original and predicted labels for the test images (as depicted in Fig. 6),

```
Original Label:  Tomato-Bacterial_Spot
Predicted Label:  Tomato-Bacterial_Spot
[2.8398555e-09 7.5014716e-05 9.9992496e-01]
```

Fig. 6. Original label and Predicted label.

The label names on the left correspond to the original labels and the names on the right represent the predicted labels (as displayed in Fig. 7). All the original labels and the predicted labels match with each other since, the model has achieved a high accuracy.

```
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Potato-Early_blight - Potato-Early_blight
Potato-Early_blight - Potato-Early_blight
Potato-Early_blight - Potato-Early_blight
Potato-Early_blight - Potato-Early_blight
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Corn-Common_rust - Corn-Common_rust
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Potato-Early_blight - Potato-Early_blight
Corn-Common_rust - Corn-Common_rust
Corn-Common_rust - Corn-Common_rust
Potato-Early_blight - Potato-Early_blight
Corn-Common_rust - Corn-Common_rust
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Corn-Common_rust - Corn-Common_rust
Potato-Early_blight - Potato-Early_blight
Potato-Early_blight - Potato-Early_blight
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
Potato-Early_blight - Potato-Early_blight
Tomato-Bacterial_Spot - Tomato-Bacterial_Spot
```

Fig. 7. Original label V/S Predicted label.

Additionally, a graphical representation was designed to illustrate the progression of training and validation accuracy over the various epochs. This graph, displayed with epochs and accuracy which are on the x-axis and y-axis respectively and it demonstrates a positive correlation wherein an increase in the number of epochs corresponds to an increase in accuracy (as depicted in Fig. 8).

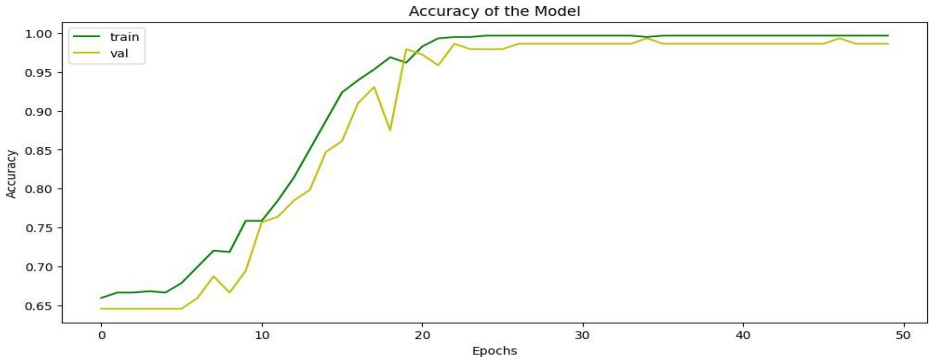

Fig. 8. Graph plotted for Epochs V/S Accuracy.

The culmination of this project is its deployment through Streamlit. Users are provided with the ability to input images of diseased leaves from potato, corn maize, and tomato plants affected by early blight, common rust, and bacterial spot respectively. Upon clicking the "Predict" button, the application promptly identifies and outputs the name of the disease the plant is afflicted with (as depicted in Fig. 9, 10, 11).

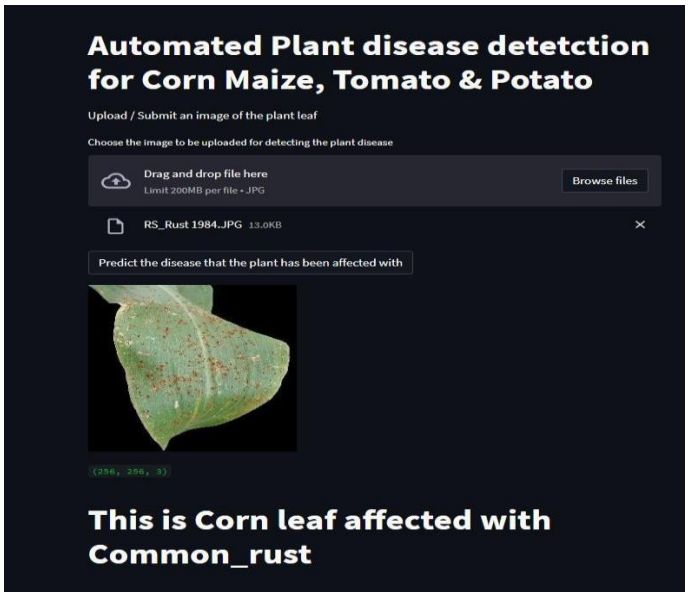

Fig. 9. Screenshot of common rust affected corn leaf deployed in Streamlit.

Fig. 10. Screenshot of early blight affected potato leaf deployed in Streamlit.

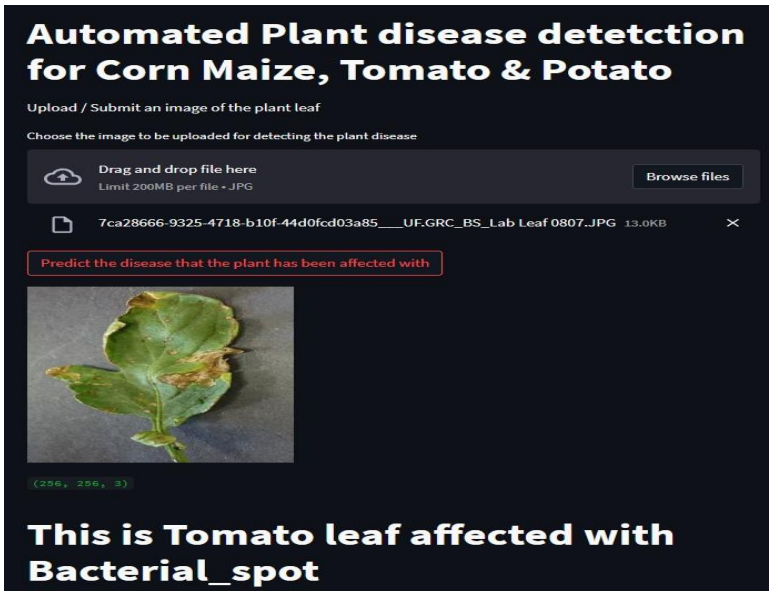

Fig. 11. Screenshot of bacterial spot affected tomato leaf deployed in Streamlit.

6 Conclusion and future work

This paper centres its focus on the prevalent diseases observed in corn maize, tomato, and potato crops, namely common rust in corn maize, early blight in potato, and bacterial spot in tomato. The primary methodology employed is a sequential convolutional neural network model, incorporating distinct layers tailored to their specific features: Conv2D, MaxPooling2D, Flatten, and Dense. The model's predictions yield remarkably accurate results, and further, it was successfully deployed in a Streamlit web application. In terms of future prospects, the project has potential for expansion. This could involve working with larger and more diverse datasets encompassing various plant types and diseases. The aim would be to improve the model's accuracy and efficiency. This iterative approach could lead to significant advancements in disease detection and could contribute positively to agricultural practices and plant health management.

References

1. Kumar, Sumit. "Plant disease detection using CNN." Turkish Journal of Computer and Mathematics Education (TURCOMAT) 12.12 (2021): 2106-2112.
2. Wallelign, Serawork, Mihai Polceanu, and Cédric Buche. "Soybean plant disease identification using convolutional neural network." FLAIRS conference. 2018.
3. P. Tm, A. Pranathi, K. SaiAshritha, N. B. Chittaragi and S. G. Koolagudi, "Tomato Leaf Disease Detection Using Convolutional Neural Networks," 2018 Eleventh International Conference on Contemporary Computing (IC3), Noida, India, 2018, pp. 1-5, doi: 10.1109/IC3.2018.8530532.
4. Bedi, Punam, and Pushkar Gole. "Plant disease detection using hybrid model based on convolutional autoencoder and convolutional neural network." Artificial Intelligence in Agriculture 5 (2021): 90-101.
5. G. Shrestha, Deepsikha, M. Das and N. Dey, "Plant Disease Detection Using CNN," 2020 IEEE Applied Signal Processing Conference (ASPCON), Kolkata, India, 2020, pp. 109-113, doi: 10.1109/ASPCON49795.2020.9276722
6. Hassan, Sk Mahmudul, et al. "Plant disease identification using shallow convolutional neural network." Agronomy 11.12 (2021): 2388.
7. Agarwal, Mohit, Suneet Kr Gupta, and K. K. Biswas. "Development of Efficient CNN model for Tomato crop disease identification." Sustainable Computing: Informatics and Systems 28 (2020): 100407.
8. Ji, Miaomiao, Lei Zhang, and Qiufeng Wu. "Automatic grape leaf diseases identification via UnitedModel based on multiple convolutional neural networks." Information Processing in Agriculture 7.3 (2020): 418-426.
9. Arivazhagan, S., and S. Vineth Ligi. "Mango leaf diseases identification using convolutional neural network." International Journal of Pure and Applied Mathematics 120.6 (2018): 11067- 11079.
10. Islam, Md Ashiqul, et al. "An automated convolutional neural network based approach for paddy leaf disease detection." International Journal of Advanced Computer Science and Applications 12.1 (2021).